

Version 1.2

August 2026



MACHINE LEARNING

MODUL 5 - TRANSFER LEARNING

Authored by:

Ir. Yufis Azhar, S.Kom., M.Kom.

Haidar Dimas Heryanto

Muhammad Aunul Hakim

INFORMATICS LABORATORY

UNIVERSITY OF MUHAMMADIYAH MALANG

TABLE OF CONTENTS

TABLE OF CONTENTS.....	1
INTRODUCTION.....	3
OBJECTIVES.....	3
MODULE OBJECTIVES.....	3
PREPARATION.....	3
MAIN TOPICS.....	4
A. TRANSFER LEARNING.....	4
B. TRANSFER LEARNING ON VARIOUS DATA TYPES.....	4
SIMPLE SIMULATION.....	7
A. IMAGE DATA CLASSIFICATION USING TRANSFER LEARNING.....	7
B. TEXT DATA CLASSIFICATION USING TRANSFER LEARNING.....	11
C. TABULAR DATA CLASSIFICATION USING TRANSFER LEARNING.....	14
CODELAB (10%).....	16
LAB ASSIGNMENT (90%).....	17
A. TASK 1 - IMAGE CLASSIFICATION.....	17
B. TASK 2 - TEXT CLASSIFICATION.....	17
C. TASK 3 - TABULAR CLASSIFICATION.....	18



INTRODUCTION

OBJECTIVES

This module explains transfer learning and how to apply it to various types of data in machine learning. Students will understand the benefits and techniques of applying transfer learning, as well as how to implement it with example datasets.

MODULE OBJECTIVES

Students will be able to understand the module's concepts, build models with pre-trained transfer learning models on various datasets, and complete practical module activities.

PREPARATION

1. Muslim students may recite the following prayer before starting the lesson

رَضِيتُ بِاللّٰهِ رَبًّا وَبِالْإِسْلَامِ دِينًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُولًا. رَبِّ زِدْنِي عِلْمًا وَارْزُقْنِي فَهْمًا

2. Google Collaboratory.
3. Library NumPy, Pandas, Matplotlib, Sklearn, and Tensorflow.
4. Source Code : [🔗 MODULE 5 MACHINE LEARNING - TRANSFER LEARNING](#)



MAIN TOPICS

In recent years, transfer learning has become one of the most popular and effective approaches in machine learning and deep learning, especially for tasks involving image, text, and speech processing. This concept emerged to address major challenges in machine learning model development: data limitations and training time.

A. TRANSFER LEARNING

Transfer learning is a machine learning technique where a model trained on one task is reused for a different but related task. Instead of training a model from scratch on new data, transfer learning lets us leverage the knowledge the model gained from previous datasets. This technique is useful when we have limited data or want to speed up training.

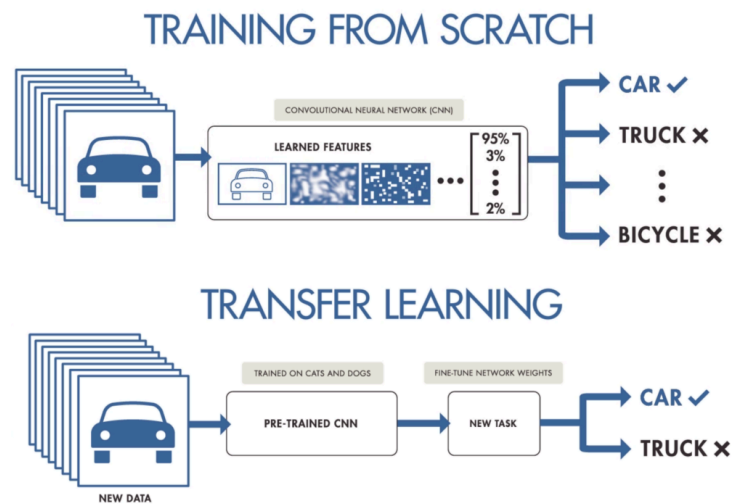


Figure 1. Comparison between training from scratch and transfer learning approaches in CNNs

Source: [Transfer Learning with Practical Implementation by Ankit Malik | Medium](https://medium.com/@ankitmalik1998/transfer-learning-with-practical-implementation-by-ankit-malik-1e1e1e1e1e1e)

In deep learning, especially with CNN (Convolutional Neural Networks) architectures, transfer learning often involves taking models trained on large datasets such as ImageNet and adapting them for more specific tasks. This process not only saves computational time and resources but can also produce more accurate models compared to training from scratch.

B. TRANSFER LEARNING ON VARIOUS DATA TYPES

Transfer learning is not limited to image data; it also applies to other data types, such as text and tabular data. Researchers have developed pre-trained models for each



data type that are available for reuse. Below is an introduction to some commonly used pre-trained models for each data type.

1. Transfer Learning on Image Data

For image data, transfer learning typically uses CNN (Convolutional Neural Network) models trained on large datasets like ImageNet. Some popular pre-trained models include:

- a) VGGNet: A model known for its simple yet effective architecture. VGGNet has various variants, such as VGG16 and VGG19, which indicate the number of layers used.
- b) ResNet: A model that introduces the concept of residual connections to overcome the degradation problem often encountered in very deep networks. ResNet has several variants such as ResNet50, ResNet101, and ResNet152.
- c) Inception: Known for its modular architecture that allows the model to capture features at various scales. This model is also known as GoogLeNet.
- d) EfficientNet: A model that optimizes computational efficiency and performance, with various versions scaled from B0 to B7.

2. Transfer Learning on Text Data

For text data, transfer learning typically involves neural network-based models such as RNNs (Recurrent Neural Networks) or Transformers. Some commonly used pre-trained models include:

- a) BERT (Bidirectional Encoder Representations from Transformers): A Transformer model trained bidirectionally, which makes it very powerful in understanding the context of words in a sentence.
- b) GPT (Generative Pre-trained Transformer): A model trained to understand text and generate new coherent text based on the given input. GPT is often used in tasks such as automatic writing and chatbots.
- c) RoBERTa (A Robustly Optimized BERT Pretraining Approach): A variant of BERT that has been optimized for better performance in various NLP tasks.
- d) XLNet: A combination of autoregressive and auto-encoding models designed to handle text sequences better.

3. Transfer Learning on Tabular Data

For tabular data, transfer learning is still a developing area of research, but several approaches have been successfully implemented:



- a) TabNet: A model that utilizes attention techniques to select the most relevant features from tabular data, enabling transfer learning on similar datasets.
- b) CatBoost, XGBoost, LightGBM: Although better known as boosting models, they can be used in transfer learning through fine-tuning techniques, where models trained on large and similar datasets can be adapted to new datasets.



SIMPLE SIMULATION

A. IMAGE DATA CLASSIFICATION USING TRANSFER LEARNING

First, let's import the necessary libraries.

```
# Import Library

import tensorflow as tf
from tensorflow import keras
import numpy as np
import matplotlib.pyplot as plt
```

Next, download the image dataset (for this simulation, we will use a cats-and-dogs dataset).

```
# Download Dataset

!wget --no-check-certificate \
https://storage.googleapis.com/mledu-datasets/cats_and_dogs_filtered.zip \
-O /tmp/cats_and_dogs_filtered.zip
```

After the download completes, extract the dataset and define the training and validation paths.

```
# Ekstrak Dataset

import zipfile
local_zip = '/tmp/cats_and_dogs_filtered.zip'
zip_ref = zipfile.ZipFile(local_zip, 'r')
zip_ref.extractall('/tmp')
zip_ref.close()

# Definisikan Path untuk Data Train dan Validation

import os
base_dir = '/tmp/cats_and_dogs_filtered'
train_dir = os.path.join(base_dir, 'train')
validation_dir = os.path.join(base_dir, 'validation')
```

Once the paths are defined, the data is ready to use. Let's first display a sample of the data.



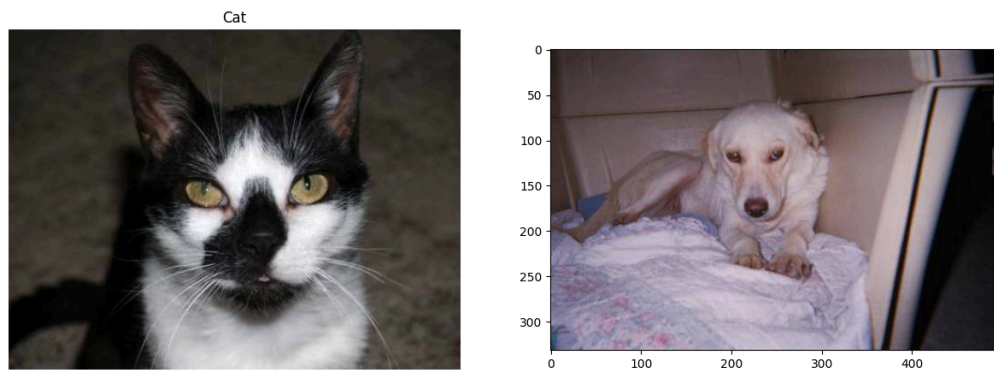
```
# Menampilkan Citra Acak dari Dataset

import random
from tensorflow.keras.preprocessing.image import load_img
fig, ax = plt.subplots(1, 2, figsize=(15, 10))

sample_cat = random.choice(os.listdir(train_dir + "/cats"))
image = load_img(train_dir + "/cats/" + sample_cat)
ax[0].imshow(image)
ax[0].set_title("Cat")
ax[0].axis("Off")

sample_dog = random.choice(os.listdir(train_dir + "/dogs"))
image = load_img(train_dir + "/dogs/" + sample_dog)
ax[1].imshow(image)
```

Result:



Perform preprocessing and augmentation.

```
# Data Augmentation

from tensorflow.keras.preprocessing.image import load_img, ImageDataGenerator
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=30,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest'
)

train_gen = train_datagen.flow_from_directory(
    train_dir,
    target_size=(150, 150),
    shuffle=True,
    batch_size=32,
    class_mode='binary'
)
```



```
datagen = ImageDataGenerator(rescale=1./255)

val_gen = datagen.flow_from_directory(
    validation_dir,
    target_size=(150, 150),
    batch_size=32,
    class_mode='binary'
)

print("Batches train = %d, validation = %d" % (len(train_gen), len(val_gen)))
```

Next, we can define the classification model. In transfer learning, we can use several techniques. These include:

- Direct Use (Directly using existing models and weights)
- Network Surgery (Using the same architecture and weights for the feature-extracted layer, but creating our own Fully Connected Layer and training specific weights for that Fully Connected Layer)
- Learning from Scratch (Using the architecture of the previous model, but not its weights. So the weights must be re-trained using our own dataset)
- A combination of all three

In this simulation, we will try to use the Network Surgery technique. We will use the Feature-Extracted Layer from a previous model (here, ResNet101), then pair it with a Fully Connected Layer that we create ourselves.

```
# Import Library yang Dibutuhkan

from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, Flatten, Dropout, Input
from tensorflow.keras.applications import ResNet101

#Load Model ResNet101, kemudian kita potong fully-connected layernya

baseModel = ResNet101(
    weights='imagenet',
    input_tensor=Input(shape=(150, 150, 3)),
    include_top=False
)
```

To train only the weights in the Fully Connected Layer, we need to freeze the Feature Extraction Layer of ResNet101 using the code below.

```
# Freeze baseModel karena sudah dilakukan training

baseModel.trainable = False
```



Next, we will define our own Fully Connected Layer. This layer will then be paired with ResNet101's feature extraction layer.

```
# Membuat Fully-connected Layer

x = baseModel.output
x = Flatten()(x)
x = Dense(256, activation='relu')(x)
x = Dropout(0.2)(x)
outputs = Dense(1, activation='sigmoid')(x)
model = Model(inputs=baseModel.input, outputs=outputs)

model.summary()
```

After that, we can compile and train the model using the dataset we downloaded earlier.

```
# Compile Model

from tensorflow.keras.optimizers import Adam
model.compile(Adam(learning_rate=0.001), loss='binary_crossentropy', metrics=['accuracy'])

# Training Transfer Learning

history = model.fit(
    train_gen,
    epochs=20,
    validation_data=val_gen,
)
```

Finally, we display the accuracy and loss plots and the model's evaluation results.

```
# Visualisasi Accuracy, Val Accuracy, Loss, dan Val Loss

import matplotlib.pyplot as plt

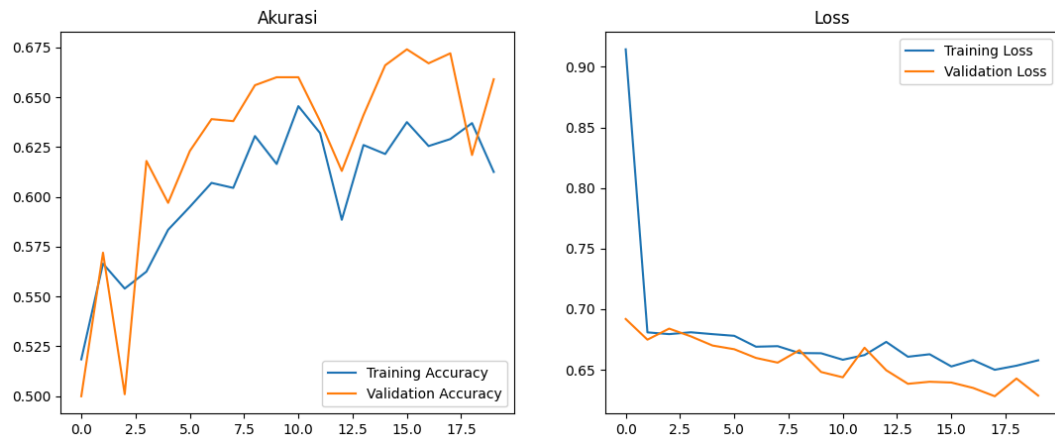
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
loss = history.history['loss']
val_loss = history.history['val_loss']
epochs_range = range(len(acc))

plt.figure(figsize=(13,5))
plt.subplot(1, 2, 1)
plt.plot(epochs_range, acc, label='Training Accuracy')
plt.plot(epochs_range, val_acc, label='Validation Accuracy')
plt.legend(loc='lower right')
plt.title('Akurasi')

plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label='Training Loss')
plt.plot(epochs_range, val_loss, label='Validation Loss')
plt.legend(loc='upper right')
plt.title('Loss')
plt.show()
```



Hasil:



```
# Menampilkan Classification Report

from sklearn.metrics import classification_report, confusion_matrix
import numpy as np

y_pred = model.predict(val_gen)
y_pred = np.argmax(y_pred, axis=1)
target_names = ['Cat', 'Dog']

print('Classification Report')
print(classification_report(val_gen.classes, y_pred, target_names=target_names))
```

32/32 ————— 16s 294ms/step

Classification Report		precision	recall	f1-score	support
	Cat	0.50	1.00	0.67	500
	Dog	0.00	0.00	0.00	500
accuracy				0.50	1000
macro avg		0.25	0.50	0.33	1000
weighted avg		0.25	0.50	0.33	1000

B. TEXT DATA CLASSIFICATION USING TRANSFER LEARNING

In this simulation, we will use BERT for Text Data Classification. Let's start by installing the Transformer library.

```
# Install Library Transformers

!pip install transformers
```



Next, import the necessary libraries and create a simple dataset for this simulation.

```
# Import Library yang Dibutuhkan

import pandas as pd
import tensorflow as tf
from transformers import BertTokenizer, TFBertForSequenceClassification
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

# Dataset
data = [
    ("This is a positive sentence.", 1),
    ("I didn't like this movie.", 0),
    ("The weather is great today.", 1),
    ("The food was terrible.", 0),
    ("I love this, sounds like Disney song", 1),
    ("It is gross, I don't like this", 0),
    ("You break my heart, break my hope", 0),
    ("So bad and it is not good for my mental health", 0),
    ("I like it, so amazing point", 1),
    ("Love the sentences so much, so good.", 1)
]

texts, labels = zip(*data)
```

Perform sufficient preprocessing, including data splitting and tokenization.

```
# Splitting Dataset Menjadi Data Training dan Test

train_texts, test_texts, train_labels, test_labels = train_test_split(texts, labels,
test_size=0.2, random_state=111)

# Menggunakan Tokenizer BERT

tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
train_encodings = tokenizer(train_texts, truncation=True, padding=True, max_length=128)
test_encodings = tokenizer(test_texts, truncation=True, padding=True, max_length=128)
```

Next, create a TensorFlow dataset.

```
# Membuat Dataset Tensorflow

train_dataset = tf.data.Dataset.from_tensor_slices(
    (dict(train_encodings),
     train_labels))

test_dataset = tf.data.Dataset.from_tensor_slices(
    (dict(test_encodings),
     test_labels))
```



After creating the dataset, load the BERT model and train it on the dataset.

```
# Memuat Model BERT untuk klasifikasi teks (contoh: sentimen positif atau negatif)

from transformers import BertConfig

# Load configuration
from transformers import TFBertForSequenceClassification, BertTokenizerFast, AutoConfig
tokenizer = BertTokenizerFast.from_pretrained("bert-base-uncased")
config = AutoConfig.from_pretrained("bert-base-uncased", num_labels=2)

# Load model from configuration
model = TFBertForSequenceClassification.from_pretrained("bert-base-uncased",
config=config, from_pt=True) # konversi dari PT
model.summary()
```

```
# Proses Training

optimizer = tf.keras.optimizers.Adam(learning_rate=1e-5)
model.compile(optimizer='adam', loss=model.hf_compute_loss, metrics=['accuracy'])
model.fit(train_dataset.shuffle(100).batch(16), epochs=20, batch_size=16)
```

Terakhir, tampilkan hasil evaluasi modelnya.

```
# Evaluasi Model

loss,accuracy = model.evaluate(test_dataset.batch(16))
print(f"Loss: {loss}, Accuracy: {accuracy}")
```

```
# Prediksi Model

predictions = model.predict(test_dataset.batch(16))
predicted_classes = tf.argmax(predictions.logits, axis=1)
```

```
# Tampilkan Classification Report

target_names = ['Negative', 'Positive']
print(classification_report(test_labels, predicted_classes, target_names=target_names))
```

	precision	recall	f1-score	support
Negative	0.00	0.00	0.00	1
Positive	0.50	1.00	0.67	1
accuracy			0.50	2
macro avg	0.25	0.50	0.33	2
weighted avg	0.25	0.50	0.33	2



C. TABULAR DATA CLASSIFICATION USING TRANSFER LEARNING

For Tabular Data Classification, we will use the TabNet model. First, import the necessary libraries. Also, create a simple dataset for this simulation.

```
# Import Library

import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

# Collect Data

data = {
    'X1': [1.2, 2.0, -0.5, -2.0, 3.0, 0.5, -1.0, 2.5],
    'X2': [0.8, 3.5, 2.0, -1.5, -1.0, -2.5, 1.0, -3.0],
    'label': [0, 0, 1, 1, 0, 1, 1, 0]
}
```

Make sure the TabNet library is installed. If not, use the following command to install it.

```
!pip install pytorch-tabnet
```

Once the dataset and libraries are ready, you can train the model directly.

```
df = pd.DataFrame(data)
train_df, test_df = train_test_split(df, test_size=0.25, random_state=42)

X_train = train_df.drop('label', axis=1).values
y_train = train_df['label'].values
X_test = test_df.drop('label', axis=1).values
y_test = test_df['label'].values

# Menggunakan Library TabNet

from pytorch_tabnet.tab_model import TabNetClassifier

clf = TabNetClassifier()
clf.fit(
    X_train=X_train,
    y_train=y_train,
    eval_set=[(X_train, y_train), (X_test, y_test)],
    eval_name=['train', 'valid'],
    eval_metric=['accuracy'],
    max_epochs=50,
    patience=10
)
```



epoch 0	loss: 0.0	train_accuracy: 1.0	valid_accuracy: 0.5	0:00:00s
epoch 1	loss: 0.0	train_accuracy: 1.0	valid_accuracy: 0.5	0:00:00s
epoch 2	loss: 0.0	train_accuracy: 1.0	valid_accuracy: 0.5	0:00:00s
epoch 3	loss: 0.0	train_accuracy: 1.0	valid_accuracy: 0.5	0:00:00s
epoch 4	loss: 0.0	train_accuracy: 1.0	valid_accuracy: 0.5	0:00:00s
epoch 5	loss: 0.0	train_accuracy: 1.0	valid_accuracy: 0.5	0:00:01s
epoch 6	loss: 0.0	train_accuracy: 1.0	valid_accuracy: 0.5	0:00:01s
epoch 7	loss: 0.0	train_accuracy: 1.0	valid_accuracy: 0.5	0:00:01s
epoch 8	loss: 0.0	train_accuracy: 1.0	valid_accuracy: 0.5	0:00:01s
epoch 9	loss: 0.0	train_accuracy: 1.0	valid_accuracy: 0.5	0:00:01s
epoch 10	loss: 0.0	train_accuracy: 1.0	valid_accuracy: 0.5	0:00:01s

Finally, display the result.

```
# Konversi Prediksi Menjadi Sebuah Label (0 atau 1)

predicted_labels = clf.predict(X_test)

# Hasil Prediksi

print("Hasil prediksi: ")
print(predicted_labels)
```

```
Hasil prediksi:
[0 0]
```

```
# Kalkulasi dan Menampilkan Classification Report

target_names = ['Class 0', 'Class 1']
print(classification_report(y_test, predicted_labels, target_names=target_names))
```

	precision	recall	f1-score	support
Class 0	0.50	1.00	0.67	1
Class 1	0.00	0.00	0.00	1
accuracy			0.50	2
macro avg	0.25	0.50	0.33	2
weighted avg	0.25	0.50	0.33	2



CODELAB (10%)

Rewrite the code explained in the simulation section above, then run it entirely. Ensure no errors occur during execution. After all the code runs smoothly, please show the results to the assistant for verification.

DETAILS OF CODELAB'S WORK

DETAIL	POINT
Image Data Classification Works Well	40
Text Data Classification Works Well	30
Tabular Data Classification Works Well	30
TOTAL	100



LAB ASSIGNMENT (90%)

Students are asked to implement the transfer learning concepts learned. This task covers three different data types: image, text, and tabular. Datasets for each data type can be downloaded via the following link:

[\[ASSIGNMENT DATASET\]](#)

A. TASK 1 - IMAGE CLASSIFICATION

1. Import all necessary libraries for data analysis and processing. (5 Points)
2. Download the citra.zip dataset from the provided link. (5 Points)
3. Extract citra.zip and set up the folder structure, including train, val, and test folders. (10 Points)
4. Split the dataset into train, val, and test folders according to the desired proportions. (10 Points)
5. Perform Exploratory Data Analysis (EDA) to understand class distribution, dataset size, and other relevant characteristics. (10 Points)
6. Apply preprocessing and data augmentation techniques you think will improve dataset efficiency for model training. (10 Points)
7. Select and implement a transfer learning model using your chosen pre-trained architecture (e.g., using TensorFlow, Keras, PyTorch, or others). (10 Points)
8. Train the model (fit) with a specified number of epochs. (10 Points)
9. Evaluate model performance by displaying accuracy and loss plots, and a classification report to see the model's performance on test data. (10 Points)
10. Select 10 random images from the split test data. (10 Points)
11. Make predictions using the trained model. (10 Points)

B. TASK 2 - TEXT CLASSIFICATION

1. Import all necessary libraries for data processing, analysis, and model implementation. (10 Points)
2. Load the dataset for analysis and model training. (10 Points)
3. Explore the dataset to understand its characteristics, including class distribution, size, and key patterns. (10 Points)
4. Apply appropriate preprocessing and data augmentation techniques to prepare the dataset for efficient model training. (10 Points)
5. Select and implement a transfer learning model. Use BERT (Bidirectional Encoder Representations from Transformers) or another model suitable for this classification task. (10 Points)
6. Train the model (fit) with a specified number of epochs, using the pre-processed dataset. (10 Points)



7. Evaluate model performance by displaying accuracy and loss plots, and a classification report to see the model's performance on test data. (10 Points)
8. Make predictions on the split test data. (10 Points)
9. Compare prediction results with true labels, and analyze model performance based on these results. (10 Points)
10. Show the model evaluation results, including plots, classification report, and comparison of true labels with predicted labels, to the assistant. (10 Points)

C. TASK 3 - TABULAR CLASSIFICATION

1. Import all necessary libraries for data processing, analysis, and model implementation. This includes libraries that support implementing TabNet or other transfer learning models. (10 Points)
2. Load the dataset for analysis and model training. Ensure the dataset is in the appropriate format. (10 Points)
3. Explore the data to understand its characteristics. This analysis may include class distribution, dataset size, missing values, and other important patterns that can affect model results. (10 Points)
4. Apply necessary pre-processing techniques, such as normalization, encoding, or handling missing values. Perform data augmentation if necessary to enrich the dataset and improve model performance. (10 Points)
5. Select and implement a transfer learning model using TabNet or another model you deem suitable for this task. (10 Points)
6. Be sure to understand the architecture of the model used and adapt it to the task's requirements. (10 Points)
7. Train the model (fit) with a specified number of epochs. Use the processed and augmented dataset to train the model. (10 Points)
8. Evaluate model performance by displaying accuracy and loss plots during training, as well as a classification report that includes metrics such as precision, recall, and F1-score. (10 Points) Make predictions on the pre-split test data.
9. Compare the predictions with the true labels and analyze any differences or consistencies. (10 Points)
10. Show the model evaluation results, including plots, classification report, and a comparison between true labels and predicted labels, to the assistant. (10 Points)



FINAL GRADE WEIGHTING

Activity	Description	Weight
Codelab		10%
Lab Assignment	Task 1 - Image Classification	15%
	Task 2 - Text Classification	15%
	Task 3 - Tabular Classification	15%
Explanation to Assistant		45%
Total		100%

